

CSC1031

OOP

Exception Handling

An **exception** is an event that disrupts the normal flow of a program.

It occurs when something **unexpected** things happen.

```
class Test {  
    public static void main(String[] args)  
    {  
        int n = 10;  
        int m = 0;  
  
        int ans = n / m;  
  
        System.out.println("Answer: " + ans);  
    }  
}
```

```
kudriam2@soc-razor1:java$ java test  
Exception in thread "main" java.lang.ArithmeticException: / by zero  
    at test.main(test.java:6)  
kudriam2@soc-razor1:java$
```

Causes

- Invalid user input
- Device failure
- Loss of network connection
- Physical limitations (out-of-disk memory)
- Code errors
- Out of bound
- Null reference
- Type mismatch
- Opening an unavailable file
- Database errors
- Arithmetic errors



Exception vs **Error**

Errors are usually caused by serious problems that are outside the control of the program

- Out of memory
- System crash

it's usually not appropriate for a program to catch errors. Instead, the best course of action is usually to **log the error and exit** the program



Exception vs Error

Exceptions are used to handle errors that **can be recovered** from within the program.

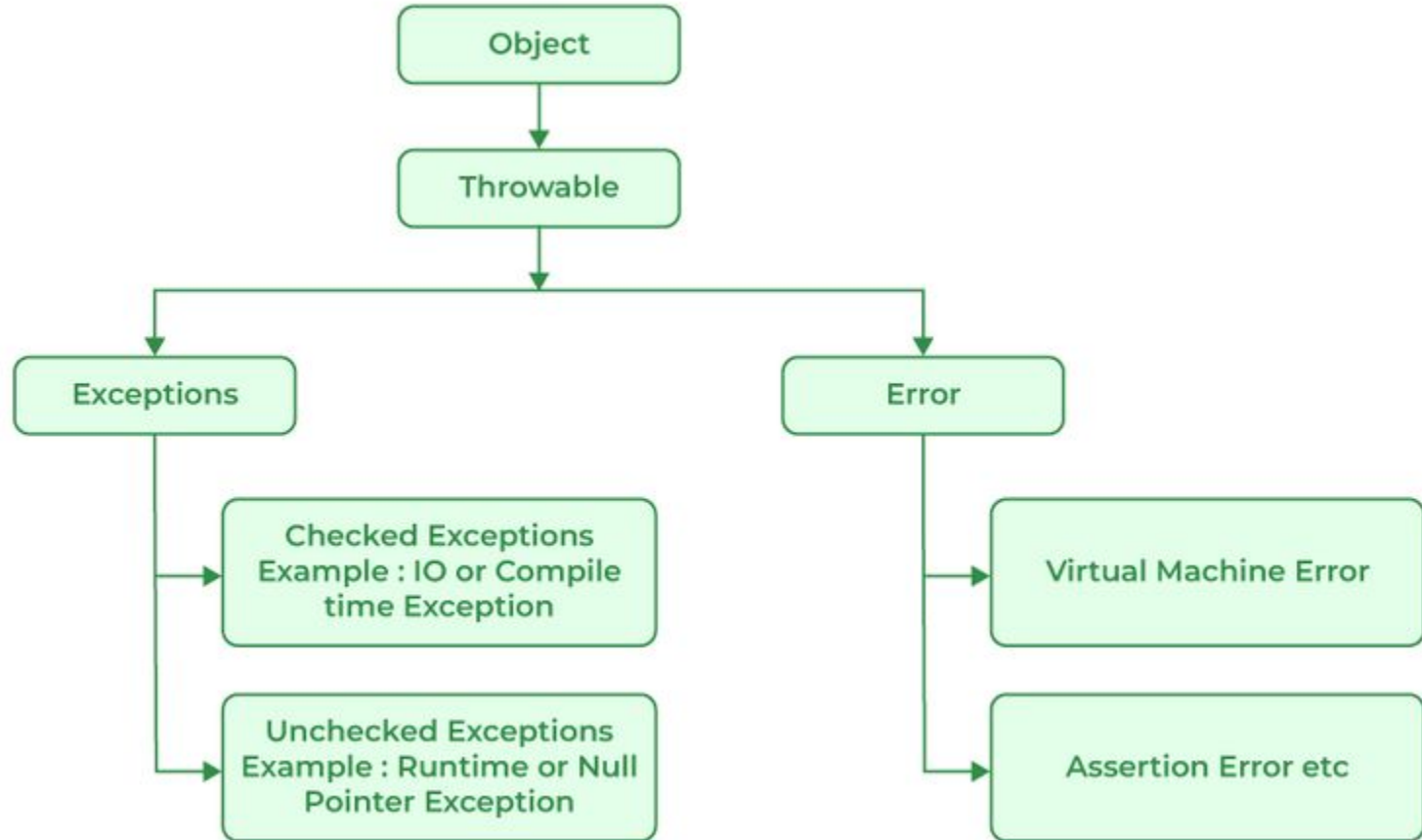
- null reference
- Input/output errors
- illegal argument is passed to a method

Exceptions **can be caught** and handled within a program, it's common to include code to catch and handle exceptions in Java programs.

By handling exceptions, you can provide more informative error messages to users and **prevent the program from crashing**.



Exception Hierarchy in Java



Checked Exceptions

Compile-time exceptions because these exceptions are checked at compile-time by the compiler.

Must be handled using **try-catch** or declared using **throws**

- IOException (File not found)
- SQLException (Database errors)
- InterruptedException (Thread interruptions)

```
try {  
    FileReader fr = new FileReader("file.txt");  
} catch (IOException e) {  
    System.out.println("File not found.");  
}
```

Unchecked Exceptions

The compiler will not check these exceptions at compile time.

Do not require explicit handling.

- `NullPointerException`
- `ArrayIndexOutOfBoundsException`
- `IllegalArgumentException`

```
int[] arr = new int[3];  
System.out.println(arr[5]);
```


Types of Exceptions

```
graph TD; A[Types of Exceptions] --> B[User-Defined Exception]; A --> C[Built-in Exception]; C --> D[Checked Exceptions]; C --> E[Unchecked Exceptions]; D --> D1[• ClassNotFoundException]; D --> D2[• InterruptedException]; D --> D3[• IOException]; D --> D4[• InstantiationException]; D --> D5[• SQLException]; D --> D6[• FileNotFoundException]; E --> E1[• ArithmeticException]; E --> E2[• ClassCastException]; E --> E3[• NullPointerException]; E --> E4[• ArrayIndexOutOfBoundsException]; E --> E5[• ArrayStoreException]; E --> E6[• IllegalStateException];
```

User-Defined Exception

Built-in Exception

Checked Exceptions

- ClassNotFoundException
- InterruptedException
- IOException
- InstantiationException
- SQLException
- FileNotFoundException

Unchecked Exceptions

- ArithmeticException
- ClassCastException
- NullPointerException
- ArrayIndexOutOfBoundsException
- ArrayStoreException
- IllegalStateException

Try-Catch Block

A try-catch block in Java is a mechanism to handle exception. The try block contains code that might throw an exception and the catch block is used to handle the exceptions if it occurs.

```
try {  
    int result = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Cannot divide by zero!");  
}
```

finally Block

The finally Block is used to execute important code regardless of whether an exception occurs or not.

Used for releasing resources (closing files, DB connections).

```
try {  
    FileReader fr = new FileReader("file.txt");  
} catch (IOException e) {  
    System.out.println("File not found.");  
} finally {  
    System.out.println("Closing resources...");  
}
```

Handling Multiple Exception

```
try {  
    // Code that may throw an exception  
  
} catch (ArithmeticException e) {  
    // Code to handle the exception  
  
} catch (ArrayIndexOutOfBoundsException e){  
    //Code to handle the another exception  
  
} catch (NumberFormatException e){  
    //Code to handle the another exception  
  
}
```

The throws Keyword

Used in method signatures to declare exceptions

```
void readFile() throws IOException {  
    FileReader fr = new FileReader("file.txt");  
}
```

Used to manually throw exceptions.

```
throw new IllegalArgumentException("Invalid input");
```

```
void checkAge(int age) throws InvalidAgeException {  
    if (age < 18) {  
        throw new InvalidAgeException("Age must be 18 or older");  
    }  
}
```

How Does JVM Handle an Exception?

1. The run-time system searches the call stack for an Exception handler
2. It starts searching from the method where the exception occurred and proceeds backward through the call stack.
3. If a handler is found, the exception is passed to it.
4. If no handler is found, the default exception handler terminates the program and prints the stack trace.

```

class Test {
    static int divideByZero(int a, int b){
        int i = a / b;
        return i;
    }

    static int computeDivision(int a, int b) {
        int res = 0;
        try {
            res = divideByZero(a, b);
        }
        catch (NumberFormatException ex) {
            System.out.println("NumberFormatException is occurred");
        }
        return res;
    }
}

```

exception
doesn't
matches

```

    public static void main(String args[]) {
        int a = 1;
        int b = 0;
        try {
            int i = computeDivision(a, b);
        }
        catch (ArithmeticException ex) {
            System.out.println(ex.getMessage());
        }
    }
}

```

appropriate
Exception
handler

Common Mistakes in Exception Handling

Catching Exception instead of specific exceptions

Swallowing exceptions (empty catch block)

Using exceptions for normal flow control

Best Practices for Writing Robust Exception Handling

Always catch the most specific exception first.

Use meaningful exception messages.

Avoid suppressing exceptions (`catch (Exception e) { }`)

Design Patterns

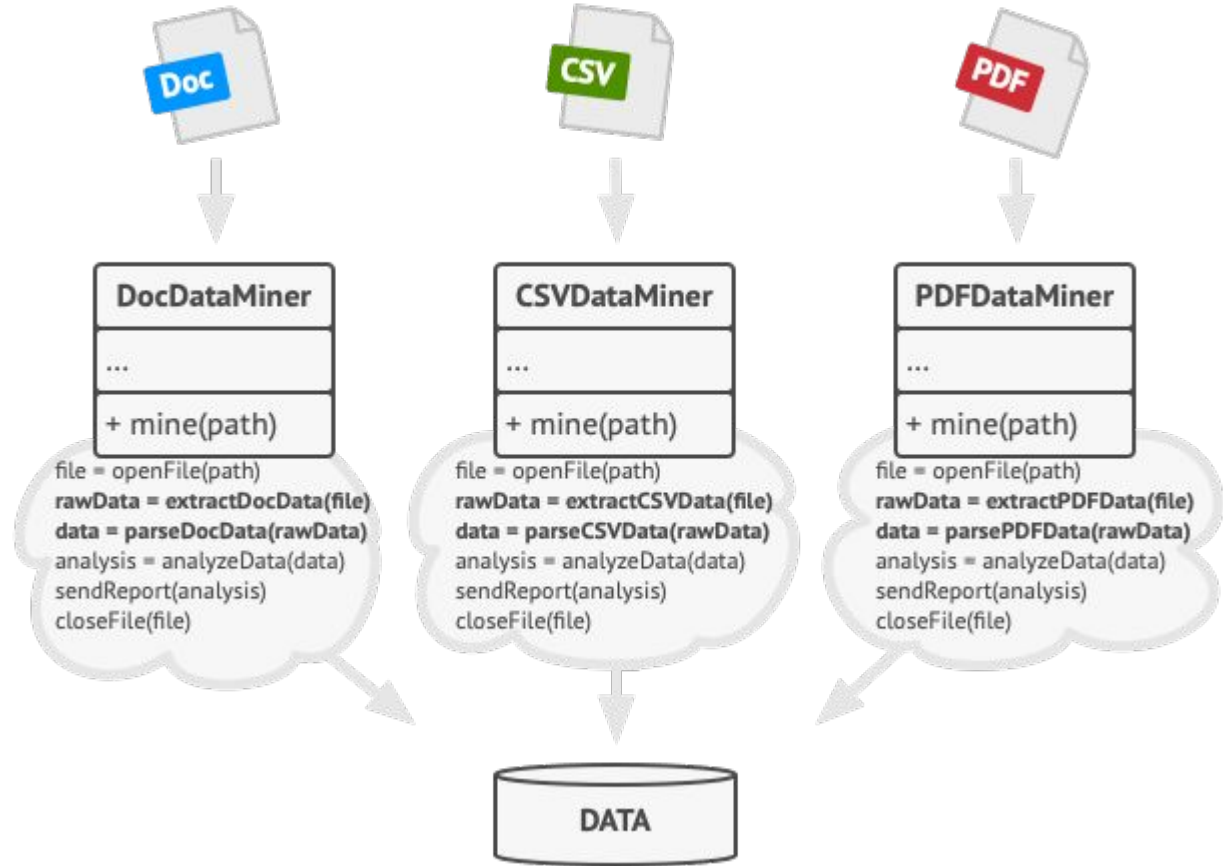
Template Method

Template Method

Template Method is a behavioral design pattern that defines the skeleton of an algorithm in the superclass but lets subclasses override specific steps of the algorithm without changing its structure

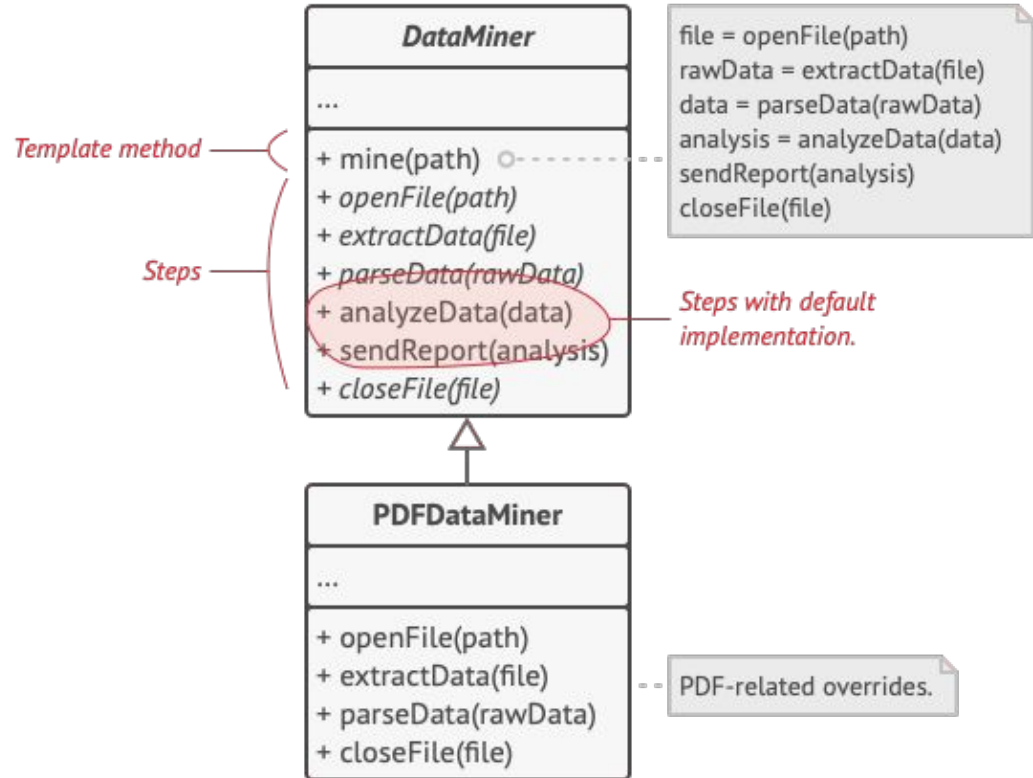
Allows subclasses to override specific steps without changing the overall algorithm structure.

Problem



Solution

The Template Method pattern suggests that you break down an algorithm into a series of **steps**, turn these steps into methods, and put a series of calls to these methods inside a single template method.

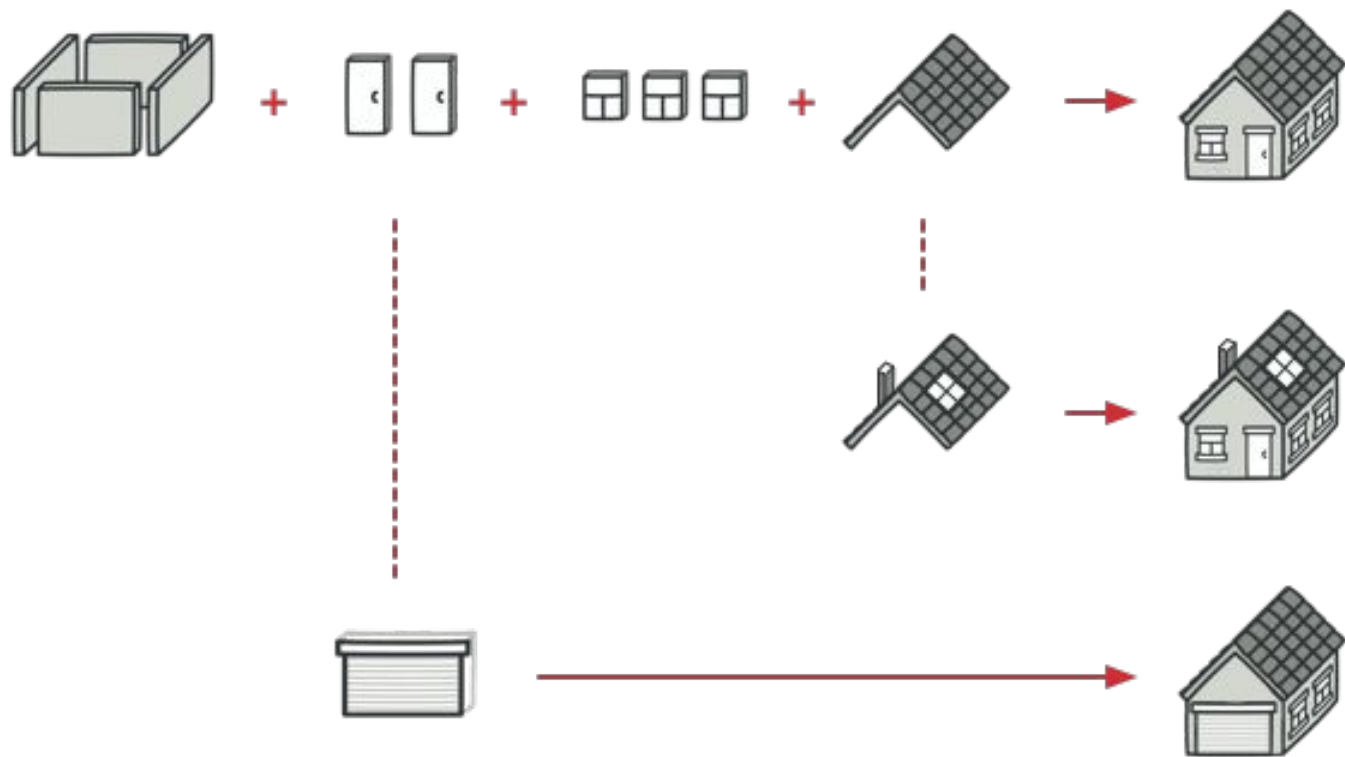


Steps

abstract steps must be implemented by every subclass

optional steps already have some default implementation, but still can be overridden if needed

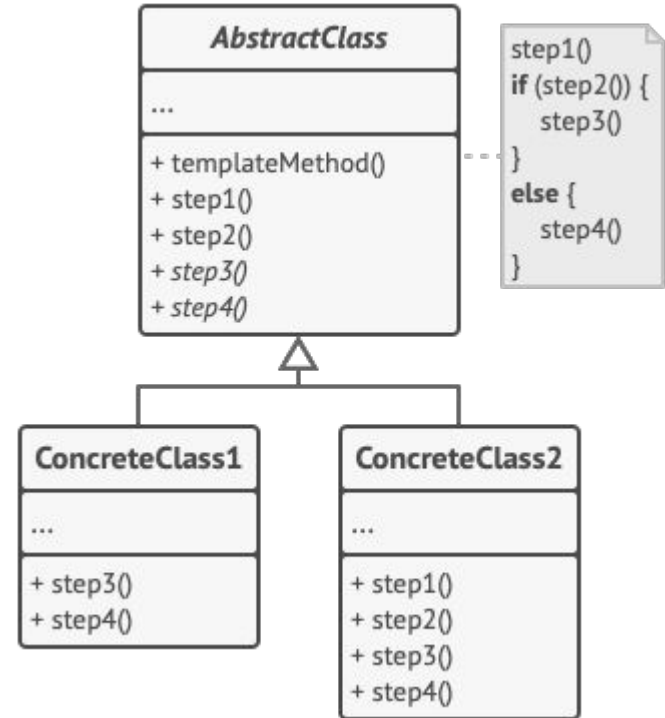
Hooks - a hook is an optional step with an empty body. A template method would work even if a hook isn't overridden. Usually, hooks are placed before and after crucial steps of algorithms, providing subclasses with additional extension points for an algorithm.



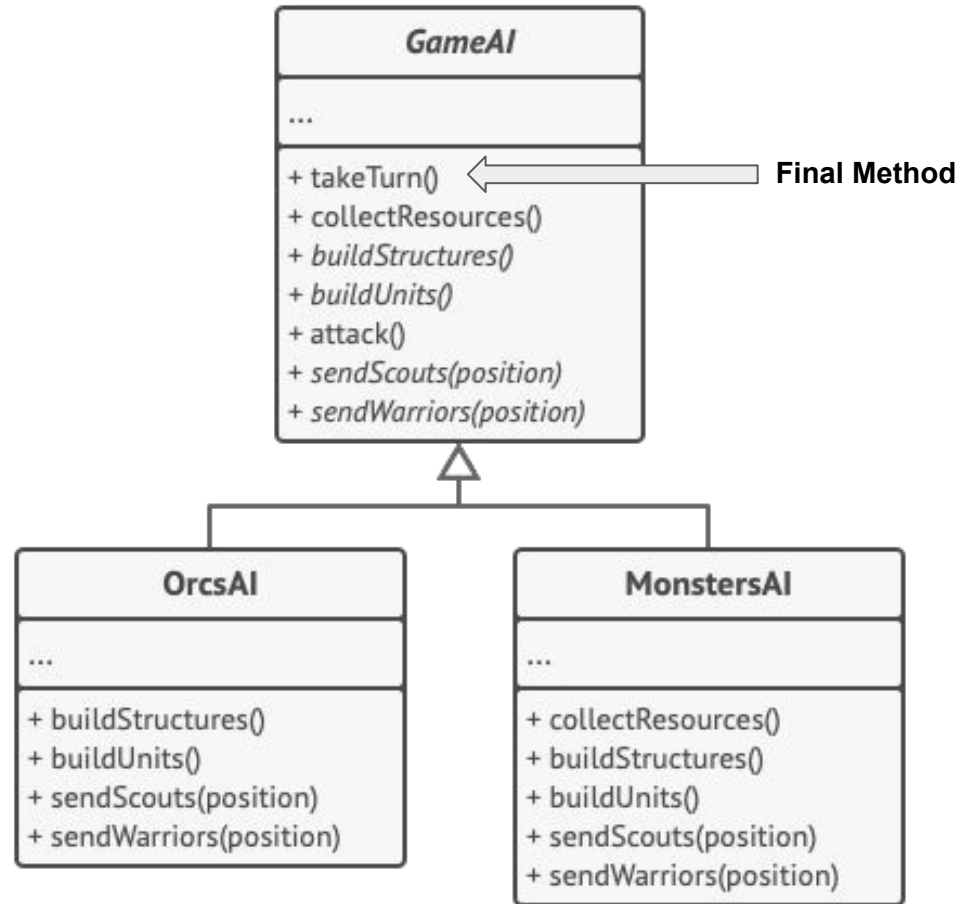
Structure

The Abstract Class declares methods that act as steps of an algorithm, as well as the actual template method which calls these methods in a specific order. The steps may either be declared abstract or have some default implementation.

Concrete Classes can override all of the steps, but not the template method itself.



Structure



Why?

- Encapsulates common behavior in a superclass.
- Reduces code duplication by enforcing a general workflow.
- Promotes the Open/Closed Principle – Base class remains unchanged while subclasses add functionality.
- Helps in framework and library development by allowing controlled customization.

When to Use the Template Method Pattern?

When **multiple classes share a common workflow**, but some steps vary.

When you want to **prevent code duplication** by enforcing a standard structure.

When you design **frameworks** and allow users to implement specific steps.

When an **algorithm should be easily extensible** with minor modifications.

How to Implement

1. Analyze the target algorithm to see whether you can **break it into steps**. Consider which steps are common to all subclasses and which ones will always be unique.
2. Create the **abstract base class** and declare the template method and a set of **abstract methods** representing the algorithm's steps. Outline the algorithm's structure in the template method by executing corresponding steps. Consider making the **template method final** to prevent subclasses from overriding it.
3. It's okay if all the steps end up being abstract. However, some steps might benefit from having a **default implementation**. Subclasses don't have to implement those methods.
4. Think of adding **hooks between the crucial steps** of the algorithm.
5. For each variation of the algorithm, create a new **concrete subclass**. It must **implement all of the abstract steps**, but may also **override** some of the optional ones.

Template Method vs Strategy Pattern

Feature	Template Method	Strategy Pattern
Code Reusability	✓ High	⚠ Depends
Algorithm Control	✓ Defined in superclass	⚠ Defined in separate classes
Flexibility	✗ Less flexible	✓ More flexible
Use Case	Standardizing workflow	Interchangeable behaviors